

A single-file, **zero-dependency**, client-side password manager built with vanilla HTML/CSS/JavaScript and the native Web Crypto API. All vault data is encrypted in the browser with a key derived from your master password — the app never sees or stores your plaintext secrets.

Cybersecurity learning project. See Limitations before using it with real accounts.

Table of Contents

- 1. Features
- 2. Getting Started
- 3. How It Works
- 4. Project Structure
- 5. Usage Guide
- 6. Security Model
- 7. Configuration
- 8. Data Storage
- 9. Accessibility
- 10. Limitations
- 11. Testing
- 12. License

1. Features

Security

- **Zero-knowledge encryption** — master password is never stored; plaintext exists only in memory while unlocked
- **PBKDF2-SHA256** key derivation with **600,000 iterations** and a unique 16-byte random salt per user
- **AES-256-GCM** authenticated encryption with a fresh random 12-byte IV on every save
- **Non-extractable keys** — the derived key cannot be exported from the browser
- **Timing-attack mitigation** on login (dummy key derivation for nonexistent usernames)
- **Exponential lockout** after 5 failed logins: 30s → 60s → 120s ... capped at 900s
- **Generic error messages** — never reveals whether a username exists
- **Clipboard auto-clear** 20 seconds after copying a password
- **Inactivity auto-lock** (5/10/15/30 min or never; default 10 min)
- **Strict Content-Security-Policy** (`default-src 'none'`)
- No hand-written cryptography — only the native Web Crypto API

Password Generator

- 8–64 character length slider with four selectable character sets
- CSPRNG (`crypto.getRandomValues`) with rejection sampling — no `Math.random()`
- Unbiased Fisher–Yates shuffle; guarantees at least one character from each selected set
- Ambiguous characters (`0/O` , `1/l/I`) excluded by default
- Live strength meter and weak-password detection (dictionary, sequences, repeats)

Vault

- Add, edit, search, filter, and delete credentials with categories (Social, Banking, Education, Work, Shopping, Other)
- Show/hide and copy passwords per entry; inline password generation
- **Reused-password detection** with badges
- **Security dashboard** with a 0–100 score ring and five health metrics
- **Audit log** (capped at 200 events + failed login attempts) — never logs secrets
- **Change master password** in place (re-encrypts the whole vault with a new salt)
- Dark/light theme (manual toggle + system preference), responsive mobile layout
- Accessible: ARIA labels, live regions, focus outlines, `prefers-reduced-motion` support

2. Getting Started

Requirements

- Any modern browser (Chrome, Edge, Firefox, Safari) with Web Crypto API support
- No installation, no dependencies, no build step, no server

Run

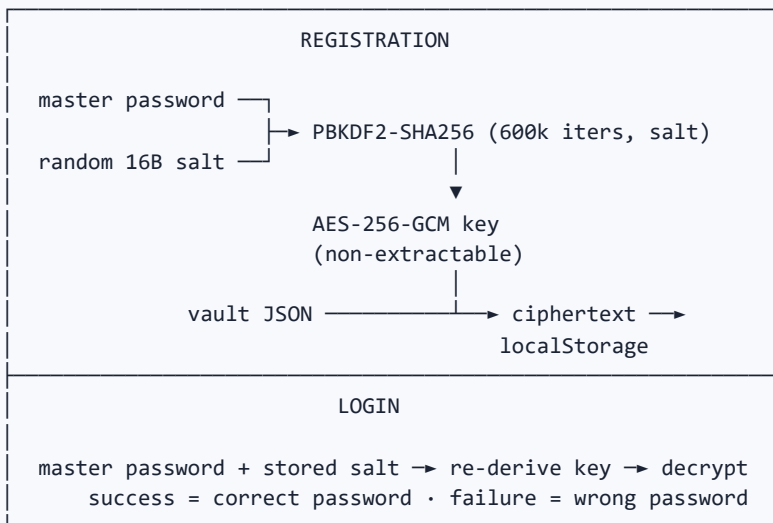
```
# Option 1 – just open the file
start "index(4).html"

# Option 2 – serve locally (optional)
python -m http.server 8080
# then open http://localhost:8080/index(4).html
```

Tip: The filename `index(4).html` suggests a duplicate download. Renaming it to `index.html` is recommended for conventional web serving:

```
Rename-Item "index(4).html" "index.html"
```

3. How It Works



1. **Register** — choose a username and a master password (≥ 12 chars, $\geq$ Medium strength). A random salt is generated and stored alongside your encrypted vault.
2. **Derive** — PBKDF2-SHA256 stretches the master password into an AES-256-GCM key that is never persisted or exported.
3. **Encrypt** — the entire vault (credentials, audit log, settings) is encrypted with a fresh IV on every save and written as ciphertext to `localStorage`.
4. **Unlock** — on login, the key is re-derived from the entered password and the stored salt. Decryption failure means the password is wrong.
5. **Auto-lock** — on timeout or logout, the key and all plaintext are cleared from memory.

4. Project Structure

```
password-manager/
└─ index(4).html    # The entire application (194 lines)
```

Internal layout of index(4).html

Lines	Section	Responsibility
1–7	Document head	Meta tags, CSP policy, title
8–47	Styles	CSS variables, light/dark themes, responsive layout, dialogs, toasts
53–76	Crypto helpers	Key derivation, encrypt/decrypt, CSPRNG, password generator, strength scoring
77–81	Storage layer	<code>localStorage</code> read/write, app state, save, audit logging
82–104	Auth & session	Register, login with lockout, lock/unlock, activity listeners
105–113	UI utilities	<code>h()</code> DOM builder, toasts, strength meter, clipboard, date formatting
114–126	Auth view	Login/register screens with live strength meter
128–131	App shell	Sidebar navigation and theme toggle
132–147	Credential dialog	Add/edit form with validation and inline generation
148–162	Vault view	Searchable, filterable credential list with reuse badges
163–167	Generator view	Interactive password generator
168–173	Security view	Score ring, stats, recent events
174–175	Audit view	Activity and failed-login table
176–185	Settings view	Auto-lock interval, master password change
186–188	Concepts view	Built-in educational page on the security concepts used
189–191	Bootstrap	Root renderer and theme initialization

5. Usage Guide

Getting started

1. Open the file in a browser.

2. Switch to the **Register** tab, enter a username and a strong master password.
3. Your vault is created empty and encrypted immediately.

Adding a credential

1. Go to **Vault** → **Add credential**.
2. Fill in name, URL (must start with `http://` or `https://`), username, and password — or click **Generate** to create one.
3. Pick a category, optionally add notes, then save.

Generating a password

1. Open **Generator**.
2. Adjust the length slider and toggle character sets.
3. Click **Copy** (auto-clears after 20s) or **Use in new credential**.

Checking your security posture

- Open **Security** for a 0–100 score ring, counts of weak/reused/old passwords, and recent activity.
- Open **Audit** for a full history of vault events and failed login attempts.

Changing your master password

1. Open **Settings** → **Change master password**.
2. Verify your current password, then enter a new one (≥12 chars, ≥ Medium).
3. The vault is re-encrypted with a **new salt** — previous keys become useless.

Locking

- Click **Lock/Logout** in the sidebar, or let the inactivity timer lock automatically.

6. Security Model

Threat	Mitigation
Offline attack on stored data	600k-iteration PBKDF2 makes brute-forcing expensive
Credential stuffing / wrong guesses	Exponential lockout after 5 failures (30s → 900s cap)
Username enumeration	Identical error text; dummy key derivation for unknown users
Timing attacks on login	Dummy derivation normalizes response time
Data tampering	AES-GCM authentication tag detects any modification
IV reuse	Fresh random 96-bit IV generated on every save
Key extraction	Keys marked non-extractable by Web Crypto
Clipboard shoulder-surfing	Copied secrets cleared after 20 seconds
Unattended session	Inactivity auto-lock (default 10 min)
Malicious resource loading	<code>default-src 'none'</code> CSP; no external resources
Weak user passwords	12-char minimum + strength meter + weak-password dictionary

Cryptographic parameters

Parameter	Value
KDF	PBKDF2-SHA256
Iterations	600,000
Salt	16 random bytes (per user)
Cipher	AES-256-GCM
IV	12 random bytes (per encryption)
Key type	Non-extractable, in-memory only
RNG	<code>crypto.getRandomValues</code> (rejection sampling)

7. Configuration

All configuration is in-file — there are no environment variables.

Setting	Location	Default	Notes
<code>ITER</code>	line 54	<code>600000</code>	PBKDF2 iterations — changing this breaks existing vaults
Auto-lock interval	in-app Settings	10 min	5 / 10 / 15 / 30 / Never
Theme	in-app toggle	system preference	Persisted in <code>localStorage["pm_theme"]</code>
CSP policy	line 6 meta tag	<code>default-src 'none'</code>	Restricts all external resource loading
Weak-password dictionary	<code>COMMON</code> , line 70	10 entries	Local common-password check

8. Data Storage

Everything lives in `localStorage` under two keys:

Key	Contents
<code>pm_users</code>	<code>{ [username]: { salt, iter, fails, until, failTimes, blob } }</code> — <code>blob</code> is base64 ciphertext of the entire vault
<code>pm_theme</code>	<code>"dark"</code> or <code>"light"</code>

The decrypted vault structure:

```
{
  "items": [
    {
      "id": "uuid",
      "name": "Example",
      "url": "https://example.com",
      "user": "alice",
      "pw": "secret",
      "cat": "Work",
      "notes": "",
      "changed": 1696800000000
    }
  ],
  "log": [{ "t": 1696800000000, "e": "login" }],
  "settings": { "lock": 10 }
}
```

Plaintext never touches disk — only ciphertext is persisted.

9. Keyboard & Accessibility

- Semantic form labels and buttons throughout
- `role="alert"` for errors, `role="status"` for toasts (announced by screen readers)
- Visible focus outlines on all interactive elements
- `prefers-reduced-motion` disables animations
- Responsive layout: horizontal navigation on screens $\leq 800\text{px}$

10. Limitations

This is an **educational project**. Please note:

- A browser-only app cannot defend against malware, keyloggers, or XSS running on the same device.
- `localStorage` can be cleared by the user, browser cleanup, or a different browser profile — **export your data elsewhere** if you need backups.
- There is no synchronization, import/export, or multi-device support.
- No independent security audit has been performed.
- **Do not use it for real accounts.** For production use, prefer a vetted manager (Bitwarden, KeePass, 1Password).

11. Testing

No automated test suite exists. Recommended manual checks:

1. Register → log out → log in with the correct password.
2. Log in with a wrong password 5 times and confirm the lockout timer engages.
3. Add a credential → reload the page → confirm it persists and decrypts.
4. Copy a password and confirm the clipboard clears after 20 seconds.
5. Wait for the auto-lock timeout and confirm plaintext is cleared.
6. Change the master password and confirm old password no longer works.

12. License

No license file is included. All rights reserved by the author by default. This is an academic/cybersecurity coursework project.